SkySwallow

开封首-2026-10-20-源代码安全检测报告

开封首-2026-10-20

北京启明星辰信息安全技术有限公司

2026-11-12

1 报告汇总信息

1.1 项目摘要

本次源代码安全测试时间段：

测试工作时间段					
开始时间	2026-11-12 14:14:22	扫描用时	2:37	来源	本地上传
地址	-				

本次源代码安全测试实施人员：

本次源代码安全测试实施人员			
账号	4s1（4s1）	部门	4s
联系方式	11@123.com		

1.2 测试范围

本次源代码安全测试范围：

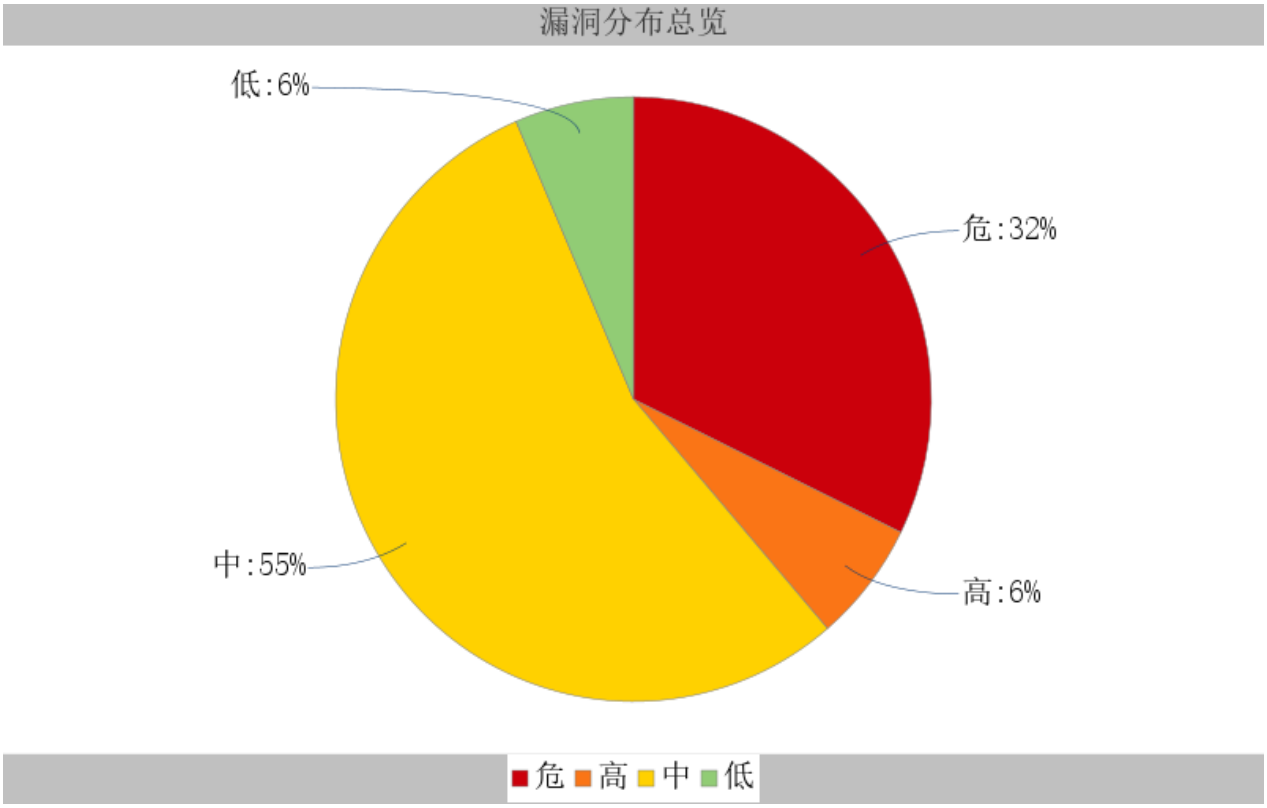
本次源代码安全测试范围			
项目名称	版本	开发语言	扫描文件数(个)
开封首	2026-10-20	GO	656
扫描项目大小(M)	千行漏洞率	漏洞数量	代码行数(行)
70.2	0.22‰	31	138822

1.3 漏洞分布

1.3.1漏洞风险汇总表

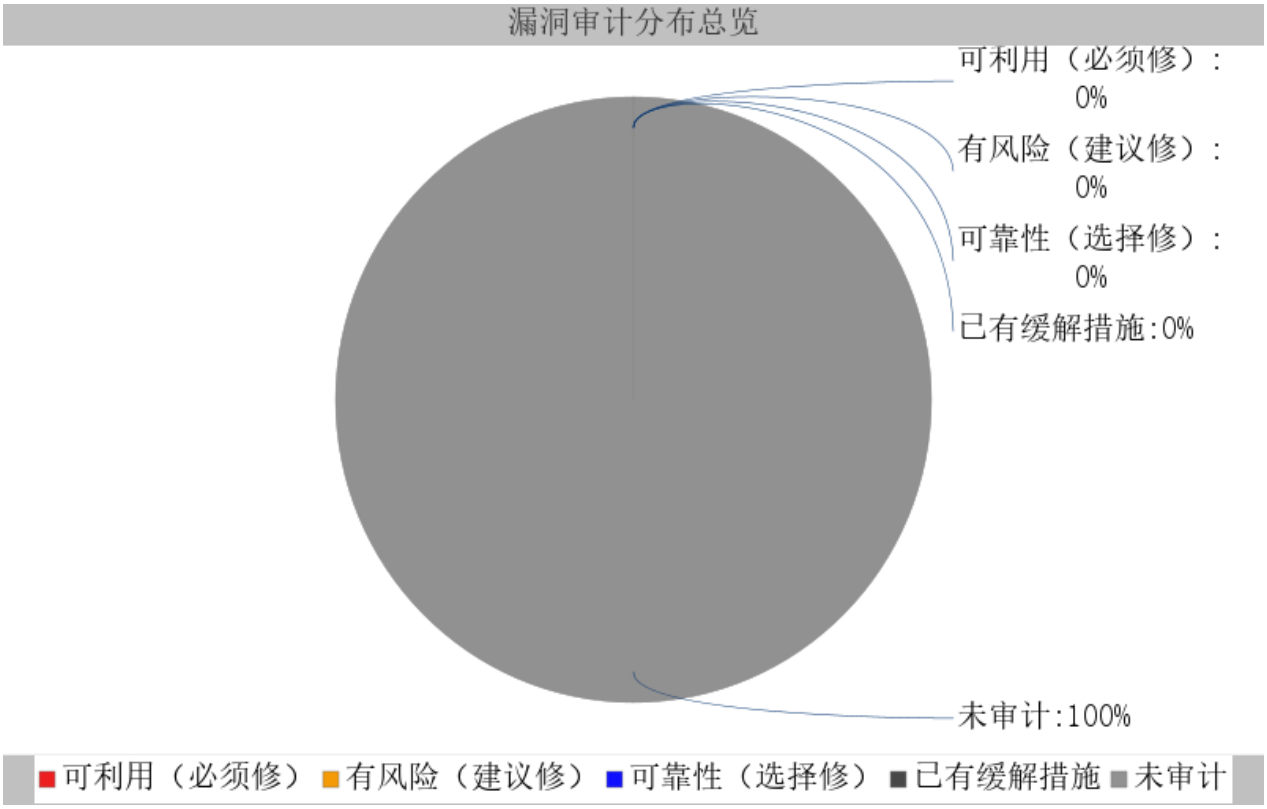
风险类别	漏洞数量
危	10
高	2
中	17
低	2

规范	无



1.3.2漏洞审计汇总

审计状态	漏洞数量
可利用（必须修）	0
有风险（建议修）	0
可靠性（选择修）	0
已有缓解措施	0
未审计	31



2 问题类别汇总清单

漏洞类别		漏洞数量
危	密钥管理：硬编码加密密钥	10
高	Cookie安全：过于宽泛的路径	2
中	不当的加密算法：未使用国家商用密码	13
中	密码管理：硬编码密码	1
中	私密违反：硬编码URL	1
中	系统信息泄漏：外部	2
低	Cookie 安全：过于宽泛的 SameSite 属性	2

3 检测问题明细

风险等级：危　　漏洞数量：10

漏洞： 密钥管理：硬编码加密密钥（10）

解释说明

硬编码加密密钥可能会削弱安全性，一旦出现安全问题将无法轻易修正。
请勿对加密密钥进行硬编码，因为这样所有项目开发人员都能查看该加密密钥，而且还会大大增加解决问题的难度。一旦代码投入使用，要更改加密密钥，就必须进行软件修补。如果受加密密钥保护的帐户被盗用，系统所有者将必须在安全性和可用性之间做出选择。

示例 1：以下代码使用硬编码加密密钥：

```
...
key := []byte("lakdsljkalkjlksd");
block, err := aes.NewCipher(key)
...
```

任何可访问该代码的人都能访问加密密钥。应用程序一经发布，除非对程序进行修补，否则将无法更改加密密钥。雇员可以利用手中掌握的信息访问权限入侵系统。如果攻击者可以访问应用程序的可执行文件，他们就可以提取加密密钥值。

修复建议

加密密钥不应为空。加密密钥应加以模糊化，并在外部数据源中进行管理。在系统中的任何位置采用明文的形式存储加密密钥，会导致任何有足够权限的人均可读取并无意中误用加密密钥。
请勿对加密密钥进行硬编码，而应对加密密钥加以模糊化，并在外部资源文件中进行管理。如果在系统中采用明文的形式存储加密密钥，任何有足够权限的人即可读取加密密钥，还可能误用这些密码。
借助专门的密钥管理系统来生成、存储和分发加密密钥。像 AWS KMS、Google Cloud KMS 这类云服务提供商提供的 KMS，能帮助安全地管理密钥。
把加密密钥存储在环境变量中，在代码里通过读取环境变量来获取密钥。
将加密密钥存于配置文件中，并且对配置文件进行严格的权限管理，确保只有授权人员能够访问。

上述案例修复之后的代码如下：

```
// 从环境变量中读取加密密钥
encryptionKey := os.Getenv("ENCRYPTION_KEY")

if encryptionKey == "" {
    fmt.Println("Encryption key not set in environment variable ENCRYPTION_KEY")
    return
}

// AES-256需要32字节的密钥长度
const requiredKeyLength = 32
if len(encryptionKey) != requiredKeyLength {
    fmt.Printf("Invalid encryption key length. Expected %d bytes, got %
d\n", requiredKeyLength, len(encryptionKey))
    return
}

// 创建一个新的AES cipher块
block, err := aes.NewCipher([]byte(encryptionKey))
```

```
if err != nil {
    fmt.Println("Error creating cipher:", err)
    return
}

fmt.Println("Cipher created successfully with dynamic encryption key.")
```

漏洞明细

编号： 开封首 – 2026–10–20 – 1/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/web/app/next.config.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/web/app/next.config.ts：73

代码片段：

```
70          source: '/cap@0.0.6/:path*',
71          headers: [
72            {
73              key: 'Cache–Control',
74              value: 'public, max–age=31536000, must–revalidate',
75            },
76          ],
```

编号： 开封首 – 2026–10–20 – 2/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/web/app/next.config.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/web/app/next.config.ts：26

代码片段：

```
23  const securityHeaders = [
24    { key: 'Content–Security–Policy', value: contentSecurityPolicy },
25    { key: 'X–Content–Type–Options', value: 'nosniff' },
```

```
26      { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
27    ];
28
29    const nextConfig: NextConfig = {
```

编号： 开封首 – 2026-10-20 – 3/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/backend/store/pg/migration/000041_add_standard_terms_system_setting.up.sql

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/backend/store/pg/migration/000041_add_standard_terms_system_setting.up.sql：7

代码片段：

```
4       '{"items":[]}'::jsonb,
5       'Global standard terms and aliases'
6     WHERE NOT EXISTS (
7       SELECT 1 FROM system_settings WHERE key = 'standard_terms'
8     );
9
```

编号： 开封首 – 2026-10-20 – 4/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/backend/store/pg/migration/000041_add_standard_terms_system_setting.down.sql

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/backend/store/pg/migration/000041_add_standard_terms_system_setting.down.sql：1

代码片段：

```
1   DELETE FROM system_settings WHERE key = 'standard_terms';
2
```

编号： 开封首 – 2026–10–20 – 5/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/web/app/next.config.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/web/app/next.config.ts：25

代码片段：

```
22
23  const securityHeaders = [
24    { key: 'Content–Security–Policy', value: contentSecurityPolicy },
25    { key: 'X–Content–Type–Options', value: 'nosniff' },
26    { key: 'Referrer–Policy', value: 'strict–origin–when–cross–origin' },
27  ];
28
```

编号： 开封首 – 2026–10–20 – 6/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/web/app/next.config.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/web/app/next.config.ts：24

代码片段：

```
21  ].join('; ');
22
23  const securityHeaders = [
24    { key: 'Content–Security–Policy', value: contentSecurityPolicy },
25    { key: 'X–Content–Type–Options', value: 'nosniff' },
26    { key: 'Referrer–Policy', value: 'strict–origin–when–cross–origin' },
27  ];
```

编号： 开封首 – 2026–10–20 – 7/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： backend/ssl/panda–wiki.key

审计状态： 未审计

批注信息： 无

路径追踪：

backend/ssl/panda-wiki.key：1

代码片段：

```
1-----BEGIN RSA PRIVATE KEY-----
2MIIEpAIBAAKCAQEAxwF98nXwfyk55nM1CRwICaUlsa0/ytqxtMYkx0veUziZlP2q
3Rkh
/BtdgW6Bd8cMXIIE6BFREZNakaiyYSHR+mGy3paBhhdIW9Wr7Ndq+CHVq5Tyb
4n08rEqNRpllOPq++l+eoO24M9tQzENZTa21w+5tU6tRO0Y2efLLB8GsNPibenV
oM
```

编号： 开封首 – 2026-10-20 – 8/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/web/admin/src/utils/auth.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/web/admin/src/utils/auth.ts：2

代码片段：

```
1   const AUTH_FLAG = 'panda_wiki_authed';
2   const LEGACY_TOKEN_KEY = 'panda_wiki_token';
3
4   /** 登录态标记（非 JWT；真实令牌走 HttpOnly Cookie） */
5   export function markAuthed() {
```

编号： 开封首 – 2026-10-20 – 9/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/backend/store/pg/migration
/000032_create_system_settings.up.sql

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/backend/store/pg/migration
/000032_create_system_settings.up.sql：29

代码片段：

```
26      ),
27      'Model setting mode configuration'
28  WHERE NOT EXISTS (
29      SELECT 1 FROM system_settings WHERE key = 'model_setting_mode'
30  );
```

编号： 开封首 – 2026–10–20 – 10/10 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki_src20261112/backend/domain/user.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/backend/domain/user.go：48

代码片段：

```
45  }
46
47  // AuthUserKey echo.Context 中保存当前登录用户（id/account/role），供审计与审批读取主体
48  const AuthUserKey = "pw_auth_user"
49
50  // AuthTokenCookieName 管理端 JWT 的 HttpOnly Cookie 名（避免 localStorage 被 XSS 读取）
51  const AuthTokenCookieName = "panda_wiki_token"
```

风险等级：高　　漏洞数量：2

漏洞： Cookie安全：过于宽泛的路径 （2）

解释说明

当设置Cookie时，Path属性决定了该Cookie可以被哪些URL路径访问。如果一个Cookie的Path设置得过于宽泛（例如设为根路径/），那么这个Cookie就可以在网站的更多部分甚至是整个域名下的所有应用中被访问到。这可能会导致敏感信息泄露或增加跨站请求伪造（CSRF）攻击的风险。

一般在以下场景会出现Cookie Security: Overly Broad Path漏洞：
1.共享主机环境：在一个服务器托管多个不同应用的情况下，如果一个应用设置了过

于宽泛的Cookie路径，其他应用也可能能够访问这些Cookie，即使它们属于不同的上下文。

2.子域问题：如果一个主域下的多个子域需要不同的安全级别，但共享了过于宽泛路径的Cookie，可能导致较低安全级别的子域影响较高安全级别的子域。

3.内部框架与外部资源：如果页面加载了来自第三方的内容（如广告、嵌入的脚本等），并且这些内容可以访问具有宽泛路径的Cookie，则可能存在安全隐患。

开发人员通常将 Cookie 设置为可从根上下文路径“/”进行访问。这会使 Cookie 暴露在该域的所有 Web 应用程序下。由于 Cookie 通常包含敏感信息（如会话标识符），因此在应用程序之间共享 Cookie 可能会导致其中一个应用程序的漏洞危及其他应用程序安全。

示例 1：

假设您在 http://communitypages.example.com/MyForum 上部署了一个论坛应用程序，当用户登录该论坛时，该应用程序将使用路径 "/" 设置会话 ID Cookie。

例如：

```
cookie := http.Cookie{
  Name:    "sessionID",
  Value:   sID,
  Expires: time.Now().AddDate(0, 0, 1),
  Path:    "/",
}
...
```

假设攻击者在 http://communitypages.example.com/EvilSite 上创建了另一个应用程序，并在论坛上发布了该站点的链接。当论坛用户点击该链接时，浏览器会将 /MyForum 设置的 Cookie 发送到在 /EvilSite 上运行的应用程序。通过这种方式窃取会话 ID 后，攻击者就能够危及浏览到 /EvilSite 的任何论坛用户的帐户安全。

除了读取 Cookie 外，攻击者还可能使用 /EvilSite 进行“Cookie Poisoning 攻击”，创建自己范围过大的 Cookie，并覆盖 /MyForum 中的 Cookie。

修复建议

确保将 cookie 路径设置为具有尽可能高的限制性。

设置Cookie时应尽量精确地指定其可访问的路径，以限制它只能被必要的页面访问。

Secure标志确保Cookie仅通过HTTPS协议发送，防止通过未加密的连接传输。HttpOnly标志阻止JavaScript脚本访问Cookie，有助于防御XSS攻击。

当设置Cookie时，可以通过指定Domain属性来控制哪个子域可以访问该Cookie。避免不必要的广泛域名设置。

示例 2：以下代码显示如何针对“说明”部分中的示例将 Cookie 路径设置为“/MyForum”。

```
cookie := http.Cookie{
  Name:    "sessionID",
  Value:   sID,
```

```
  Expires: time.Now().AddDate(0, 0, 1),
  Path:    "/MyForum",
}
...
```

漏洞明细

编号： 开封首 – 2026–10–20 – 1/2 （ 高 – Cookie安全：过于宽泛的路径 ）

源码定位： backend/utils/session_cookie.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/session_cookie.go：65 –-> AssignmentStatement

代码片段：

```
62   c.SetCookie(&http.Cookie{
63       Name:     domain.AuthTokenCookieName,
64       Value:    "",
65       Path:     "/",
66       MaxAge:   –1,
67       HttpOnly: true,
68       Secure:   CookieSecure(c),
```

编号： 开封首 – 2026–10–20 – 2/2 （ 高 – Cookie安全：过于宽泛的路径 ）

源码定位： backend/utils/session_cookie.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/session_cookie.go：52 –-> AssignmentStatement

代码片段：

```
49   c.SetCookie(&http.Cookie{
50       Name:     domain.AuthTokenCookieName,
51       Value:    token,
52       Path:     "/",
53       MaxAge:   int(maxAge.Seconds()),
```

```
54      HttpOnly: true,
55      Secure:   CookieSecure(c),
```

风险等级：中　　漏洞数量：17

漏洞： 不当的加密算法：未使用国家商用密码 （ 13 ）

解释说明

商用密码是指对不涉及国家秘密内容的信息进行加密保护或者 安全认证 所使用的 密码技术 和密码产品。 商用密码技术 是商用密码的核心，是信息化时代社会团体、组织、企事业单位和个人用于保护自身权益的重要工具。 国家将商用密码技术列入国家秘密，任何单位和个人都有责任和义务保护商用密码技术的秘密。

Incorrect Cryptographic Algorithm 指的是在使用加密算法时，选择了不合适、不安全或者已被破解的加密算法，从而导致数据的机密性、完整性和可用性受到威胁。加密算法是保障数据安全的重要手段，不同的算法在安全性、性能和适用场景上有所不同。如果使用了不安全的算法，攻击者可能会通过分析密文来破解出明文，或者篡改密文而不被察觉。

缺陷示例：

```
...
//使用过时的DES算法
KeyGenerator keyGen = KeyGenerator.getInstance("DES");
SecretKey secretKey = keyGen.generateKey();

Cipher cipher = Cipher.getInstance("DES");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);

String plaintext = "Hello, World!";
byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.
UTF_8));
String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
...
```

在上述代码中，使用了过时的 DES 算法进行加密，由于 DES 算法的密钥长度较短，容易被暴力破解，存在安全风险。

修复建议

所有算法中其中的ZUC、SM1、SM2、SM3、SM4、SM7、SM9是国内认可的商密算法，其余为国际算法。截止至本文当日，SM1和SM7仍然未公开，使用SM1和SM7只能借助于具备商密资质的安全芯片。ZUC、SM2、SM3、SM4、SM9算法已经公开，网上也可找到公开的源码。实际使用中尽量使用已经公开的商密算法，其他未公开的商密算法和国际算法只有在特定需求中才使用。在商密产品认证过程中，原则上都要使用商密算法，尽量避免使用国际算法。

上述案例修复之后的代码如下：

```
                    // 使用安全的 AES 算法
        KeyGenerator keyGen = KeyGenerator.getInstance("AES");
        keyGen.init(256); // 使用 256 位密钥
        SecretKey secretKey = keyGen.generateKey();

        Cipher cipher = Cipher.getInstance("AES");
        cipher.init(Cipher.ENCRYPT_MODE, secretKey);

        String plaintext = "Hello, World!";
        byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.
UTF_8));
        String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
```

在修复后的代码中，使用了安全的 AES 算法进行加密，并将密钥长度设置为 256 位。AES 算法具有较高的安全性和性能，能够有效保护数据的机密性。

漏洞明细

编号： 开封首 – 2026–10–20 – 1/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/telemetry/sm4.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/telemetry/sm4.go：33 –-> Seal()

代码片段：

```
30      return "", err
31  }
32
33  ciphertext := gcm.Seal(nonce, nonce, data, nil)
34  return base64.StdEncoding.EncodeToString(ciphertext), nil
35}
```

编号： 开封首 – 2026–10–20 – 2/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/utils/epub.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/epub.go：143 –-> Copy()

代码片段：

```
140     e.logger.Debug("add File", "file name", clearFileName(e.resourcesIdMap
[itemRef.IDRef].Href))
141     if r, ok := res[clearFileName(e.resourcesIdMap[itemRef.IDRef].Href)]; ok {
142         result.WriteString("<span id=" + title + "></span>\n\n")
143         if _, err := io.Copy(result, r); err != nil {
144             return "", nil, err
145         }
146         result.WriteString("\n\n")
```

编号： 开封首 – 2026–10–20 – 3/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/utils/epub.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/epub.go：124 –-> Copy()

代码片段：

```
121   result := bytes.NewBuffer(nil)
122   for _, href := range p.Guide.References {
123       if r, ok := res[clearFileName(href.Href)]; ok {
124           if _, err := io.Copy(result, r); err != nil {
125               return "", nil, err
126           }
127           result.WriteString("\n\n")
```

编号： 开封首 – 2026–10–20 – 4/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：280 –-> Copy()

代码片段：

```
277    if err != nil {
278        return nil, err
279    }
280    if _, err := io.Copy(fw, tmpFile); err != nil {
281        return nil, err
282    }
```

编号： 开封首 – 2026–10–20 – 5/13 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： backend/handler/v1/backup.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/handler/v1/backup.go：143 –-> Copy()

代码片段：

```
140    c.Response().Header().Set(echo.HeaderContentType, "application/zip")
141    c.Response().Header().Set(echo.HeaderContentDisposition, fmt.Sprintf
(`attachment; filename="%s"`, name))
142    c.Response().Header().Set(echo.HeaderContentLength, fmt.Sprintf("%d",
st.Size()))
143    _, err = io.Copy(c.Response().Writer, f)
144    if err != nil {
145        return err
146    }
```

编号： 开封首 – 2026–10–20 – 6/13 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：53 –-> Copy()

代码片段：

```
50      if err != nil {
51          return nil, err
52      }
53      if _, err := io.Copy(fw, file); err != nil {
54          return nil, err
55      }
56  }
```

编号： 开封首 – 2026–10–20 – 7/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/usecase/backup.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/usecase/backup.go：420 –-> Copy()

代码片段：

```
417         return count, bytes, err
418     }
419     hasher := utils.NewStreamingSM3()
420     n, err := io.Copy(io.MultiWriter(w, hasher), reader)
421     _ = reader.Close()
422     if err != nil {
423         return count, bytes, err
```

编号： 开封首 – 2026–10–20 – 8/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/usecase/backup.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/usecase/backup.go：293 –-> WriteString()

代码片段：

```
290         _ = os.Remove(tmp)
291         return nil, err
292     }
293     if _, err := io.WriteString(sw, utils.HMACSM3Hex(u.signKey, string(raw)));
err != nil {
294         _ = zw.Close()
295         _ = f.Close()
296         _ = os.Remove(tmp)
```

编号： 开封首 – 2026–10–20 – 9/13 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：130 –-> Copy()

代码片段：

```
127         return err
128     }
129     defer out.Close()
130     _, err = io.Copy(out, resp.Body)
131     return err
132}
```

编号： 开封首 – 2026–10–20 – 10/13 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：402 –-> Copy()

代码片段：

```
399     if err != nil {
400         return err
```

```
401   }
402   if _, err := io.Copy(fw, tmpFile); err != nil {
403       return err
404   }
```

编号： 开封首 – 2026–10–20 – 11/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/utils/field_crypto.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/field_crypto.go：64 –-> Seal()

代码片段：

```
61   if _, err := rand.Read(nonce); err != nil {
62       return "", err
63   }
64   sealed := aead.Seal(nonce, nonce, []byte(plain), nil)
65   return sm4FieldPrefix + base64.StdEncoding.EncodeToString(sealed), nil
66}
```

编号： 开封首 – 2026–10–20 – 12/13 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/utils/integrity.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/integrity.go：83 –-> Seal()

代码片段：

```
80   if _, err := rand.Read(nonce); err != nil {
81       return nil, nil, err
82   }
83   return nonce, aead.Seal(nil, nonce, plain, nil), nil
84}
85
```

```
86var ErrSM4GCMOpen = errors.New("decrypt failed: wrong password or data corrupted")
```

编号： 开封首 – 2026–10–20 – 13/13 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： backend/utils/processor.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/processor.go：66 –-> Copy()

代码片段：

```
63   if node == nil {
64       return nil
65   }
66   if _, err := io.Copy(t.result, node.buf); err != nil {
67       return err
68   }
69   for _, son := range node.son {
```

漏洞： 密码管理：硬编码密码 （1）

解释说明

Hardcoded Password 会削弱系统安全性，并会导致无法轻易修正出现的安全问题。
请勿使用硬编码方式处理密码。通过硬编码方式处理密码不仅会让所有项目开发人员都可以看到密码，而且还会使解决这一问题变得极其困难。在代码投入使用之后，除非对软件进行修补，否则将无法更改密码。如果受密码保护的帐户遭受入侵，系统所有者将必须在安全性和可用性之间做出选择。

示例：以下代码对密码进行了硬编码处理：

```
pwd := "letmein"
...
response.SetBasicAuth(usrName, pwd)
```

该代码可以正常运行，但是任何有该代码访问权限的人都可以获取该密码。一旦程序发布，除非修补该程序，否则可能无法更改密码“tiger”。雇员可以利用手中掌握的信息访问权限入侵系统。更糟的是，如果攻击者能够访问应用程序的二进制码，他们就可以利用多种常用的反编译器来访问经过反汇编的代码，而在这些代码中恰恰包含着用户使用过的密码值。

修复建议

要避免此类缺陷，需要注意以下几点：

（1）通常情况下，应对密码加以模糊化，并且在外部资源文件中进行处理。

（2）系统中避免使用明文方式存储密码，对密码进行加密处理后再存储。

上述案例修复之后的代码如下：

```
package main

import (
    "fmt"
    "net/http"
    "os"
)

func main() {
    usrName := os.Getenv("DB_USERNAME")
    pwd := os.Getenv("DB_PASSWORD")

    if usrName == "" || pwd == "" {
        fmt.Println("环境变量 DB_USERNAME 或 DB_PASSWORD 未设置")
        return
    }

    resp, err := http.Get("your_url")
    if err != nil {
        fmt.Println("请求出错:", err)
        return
    }
    defer resp.Body.Close()

    req, err := http.NewRequest("GET", "your_url", nil)
    if err != nil {
        fmt.Println("创建请求出错:", err)
        return
    }
    req.SetBasicAuth(usrName, pwd)

    client := &http.Client{}
    resp, err = client.Do(req)
    if err != nil {
        fmt.Println("执行请求出错:", err)
        return
    }
    defer resp.Body.Close()

    fmt.Println("请求成功，状态码:", resp.StatusCode)
}
```

使用 os.Getenv 函数从环境变量中获取用户名和密码，检查环境变量是否正确设置，如果未设置则输出错误信息并终止程序，使用 req.SetBasicAuth 方法将从环境变量中获取的用户名和密码设置到 HTTP 请求中，创建 HTTP 请求并执行，根据返回结果处理可能出现的错误。

漏洞明细

编号： 开封首 – 2026–10–20 – 1/1 （ 中 – 密码管理：硬编码密码 ）

源码定位： PandaWiki_src20261112/backend/log/redact_test.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki_src20261112/backend/log/redact_test.go：64

代码片段：

```
61      }
62      var buf bytes.Buffer
63      newTestLogger(&buf).Info("call https://api.example.com/x?
access_token=tok123&code=c0de&page=2",
64          slog.Any("user", user{UserID: "u1", Mobile: "13900001111", Password:
"secret-pass"}),
65          slog.String("body", `{"token":"t-xyz","name":"bob"}`),
66          slog.Any("error", errors.New("dial failed for 13700002222")),
67      )
```

漏洞： 私密违反：硬编码URL （ 1 ）

解释说明

代码中存在硬编码URL地址，这很有可能是开发人员遗留的测试信息。但如果有真实的URL地址在代码注释中，那么所有开发人员都将有意无意中获取到这些敏感信息，从而产生私密信息泄露的风险。 同时，由于工作流程不合规范或者在程序发布过程中存在疏忽，这些包含了硬编码URL地址的代码一旦发布上线，存在于正式运行的系统中，则很有可能被攻击者所利用。

当参考的 URL 发生变化时，由于注释中的 URL 没有被纳入代码的管理和更新机制，开发者可能会忘记更新注释中的 URL，导致注释中的信息与实际情况不符，给后续的开发者带来困扰。

在某些行业或组织中，对代码中的敏感信息有严格的合规要求。硬编码在注释中的 URL 可能违反这些合规要求，导致潜在的法律风险。

例：以下代码中使用硬编码URL地址

String payServerUrl = "http://www.pay.com"

修复建议

不要在配置文件中配置URL地址，如果是用于测试的代码，在上线前删除掉。

如果 URL 包含敏感信息，不要将其硬编码在注释中。可以使用更安全的方式来存储和管理这些信息，如使用环境变量、配置文件等。

建立代码审查机制，定期审查代码中的注释，确保注释中的 URL 是最新的。如果 URL 发生变化，及时更新注释。

如果无法避免在注释中使用 URL，可以对 URL 进行脱敏处理，例如只保留域名部分，不显示具体的路径和参数。

上述案例修复之后的代码如下：

```
// 参考文档：https://example.com
public class ExampleClass {
    // 类的具体实现
}
```

漏洞明细

编号： 开封首 – 2026–10–20 – 1/1 （ 中 – 私密违反：硬编码URL ）

源码定位： backend/usecase/sitemap.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/usecase/sitemap.go：57 –-> StringLiteral

代码片段：

```
54
55func sitemapNamespace() string {
56    // Official sitemap protocol namespace; constructed to avoid a single
hardcoded URL literal.
57    return "http" + "://" + "www.sitemaps.org" + "/schemas/sitemap/0.9"
58}
```

漏洞： 系统信息泄漏：外部（2）

解释说明

揭示系统数据或调试信息可帮助攻击者了解系统并制定攻击计划。
当系统数据或调试信息通过套接字或网络连接使程序流向远程机器时，就会发生外部信息泄露。
示例 1：以下例子会通过 HTTP 响应泄露系统信息。

```
func handler(w http.ResponseWriter, r *http.Request) {
  host, err := os.Hostname()
  ...
  fmt.Fprintf(w, "%s is busy, please try again later.", host)
}
```

在某些情况下，该错误消息会告诉攻击者该系统易遭受的确切攻击类型有哪些。例如，数据库错误消息可以揭示应用程序容易受到 SQL Injection 攻击。其他错误消息可以揭示有关该系统的更多间接线索。在Example 1 中，泄露的信息可能会暗示有关操作系统类型、系统上安装了哪些应用程序以及管理员在配置程序时采取了哪些保护措施的信息。

修复建议

编写错误消息时，始终要牢记安全性。在编码的过程中，尽量避免使用繁复的消息，提倡使用简短的错误消息。限制生成与存储繁复的输出数据可帮助管理员和程序员诊断问题。请留意，调试踪迹有时可能出现在不明显的位置（例如，嵌入在错误页 HTML 的注释中）。

即便是并未揭示栈踪迹或数据库转储的简短错误消息，也有可能帮助攻击者发起攻击。例如，“Access Denied”（拒绝访问）消息可以揭示系统中存在一个文件或用户。

漏洞明细

编号： 开封首 – 2026–10–20 – 1/2（中 – 系统信息泄漏：外部）

源码定位： backend/telemetry/client.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/telemetry/client.go：66 –-> Getenv(return)

backend/telemetry/client.go：66 –-> TrimSpace(0 : return)

backend/telemetry/client.go：66 –-> Assignment to baseURL

backend/telemetry/client.go：74 –-> Assignment to baseURL

backend/telemetry/client.go：73 –-> Assignment to client

backend/telemetry/client.go：103 –-> reportInstallation(0.baseURL)

backend/telemetry/client.go：292 –-> NewRequest(1 : return.value_0)

backend/telemetry/client.go：292 –-> Assignment to req

backend/telemetry/client.go：299 –-> Do(1)

攻击源：

```
63
64  // NewClient creates a new telemetry client
65  func NewClient(logger *log.Logger, repo *pg.KnowledgeBaseRepository,
modelUsecase *usecase.ModelUsecase, userUsecase *usecase.UserUsecase,
nodeRepo *pg.NodeRepository, conversationRepo *pg.ConversationRepository,
mcpRepo *pg.MCPRepository, cfg *config.Config) (*Client, error) {
66      baseURL := strings.TrimSpace(os.Getenv("TELEMETRY_BASE_URL"))
67      aesKey := strings.TrimSpace(os.Getenv("TELEMETRY_AES_KEY"))
68      if aesKey == "" {
69          // backward-compatible alias
```

爆发点：

```
296
297  req.Header.Set("Content-Type", "application/json")
298
299  resp, err := c.httpClient.Do(req)
300  if err != nil {
301      return fmt.Errorf("send request: %w", err)
302  }
```

编号： 开封首 – 2026-10-20 – 2/2 （中 – 系统信息泄漏：外部）

源码定位： backend/telemetry/client.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/telemetry/client.go：66 –-> Getenv(return)

backend/telemetry/client.go：66 –-> TrimSpace(0 : return)

backend/telemetry/client.go：66 –-> Assignment to baseURL

backend/telemetry/client.go：74 –-> Assignment to baseURL

backend/telemetry/client.go：73 –-> Assignment to client

backend/telemetry/client.go：108 –-> startPeriodicReport(0.baseURL)

backend/telemetry/client.go：196 –-> reportData(0.baseURL)

backend/telemetry/client.go：391 –-> NewRequest(1 : return.value_0)

backend/telemetry/client.go：391 –-> Assignment to req

backend/telemetry/client.go：398 –-> Do(1)

攻击源：

```
63
64   // NewClient creates a new telemetry client
65   func NewClient(logger *log.Logger, repo *pg.KnowledgeBaseRepository,
modelUsecase *usecase.ModelUsecase, userUsecase *usecase.UserUsecase,
nodeRepo *pg.NodeRepository, conversationRepo *pg.ConversationRepository,
mcpRepo *pg.MCPRepository, cfg *config.Config) (*Client, error) {
66       baseURL := strings.TrimSpace(os.Getenv("TELEMETRY_BASE_URL"))
67       aesKey := strings.TrimSpace(os.Getenv("TELEMETRY_AES_KEY"))
68       if aesKey == "" {
69           // backward-compatible alias
```

爆发点：

```
395
396   req.Header.Set("Content-Type", "application/json")
397
398   resp, err := c.httpClient.Do(req)
399   if err != nil {
400       return fmt.Errorf("send request: %w", err)
401   }
```

风险等级：低　　漏洞数量：2

漏洞： Cookie 安全：过于宽泛的 SameSite 属性 （2）

解释说明

会话 Cookie 上的 SameSite 属性未设置为 SameSiteStrictMode。

SameSite 属性保护 Cookie 免受 Cross-Site Request Forgery (CSRF) 等攻击。会话 Cookie 代表用户访问站点，以便用户可以执行授权操作。但是，浏览器会自动将 Cookie 与请求一起发送，因此用户和网站会隐式信任浏览器进行授权。攻击者可以滥用此信任，通过在攻击者控制的第三方网站页面中的 link 和 iframe 等标签的 href 和 src 属性中嵌入链接，代表用户向站点发出请求。如果攻击者能够将一位毫无戒备的用户引诱到他们控制的第三方站点，则攻击者可以发出自动包含用于授权该用户的会话 Cookie 的请求，从而可以使攻击者获取有效授权，就像他们是用户一样。将会话 Cookie 设置为 SameSite 属性的 SameSiteStrictMode，这将限制浏览器仅将 Cookie 附加到顶级导航时的请求或来自同一站点的请求。来自第三方站点的请求（通过 iframe、img 和 form 等各种标签中的链接）不含这些 Cookie，因此能阻止站点采取用户可能未授权的操作。

示例 1：以下代码将对会话 Cookie 启用 SameSite 属性中的 SameSiteLaxMode。

```
c := &http.Cookie{
  Name: "cookie",
  Value: "samesite-lax",
  SameSite: http.SameSiteLaxMode,
}
```

修复建议

在 Cookie 配置中将 SameSite 属性设置为 SameSiteStrictMode。

示例 2：以下代码将对会话 Cookie 启用 SameSite 属性中的 SameSiteStrictMode。

```
c := &http.Cookie{
  Name: "cookie",
  Value: "samesite-strict",
  SameSite: http.SameSiteStrictMode,
}
```

此外，建议开发人员继续向站点添加传统的 Cross-Site Request Forgery (CSRF) 缓解以及 SameSite 属性。许多用户可能会使用旧的浏览器版本访问该站点。旧的浏览器版本不支持 SameSite 属性。

漏洞明细

编号： 开封首 – 2026-10-20 – 1/2（低 – Cookie 安全：过于宽泛的 SameSite 属性）

源码定位： backend/utils/session_cookie.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/session_cookie.go：69 –-> FieldAccess: SameSite

代码片段：

```
66      MaxAge:   –1,
67      HttpOnly: true,
68      Secure:   CookieSecure(c),
69      SameSite: http.SameSiteLaxMode,
70   })
71}
```

编号： 开封首 – 2026–10–20 – 2/2 （低 – Cookie 安全：过于宽泛的 SameSite 属性）

源码定位： backend/utils/session_cookie.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/session_cookie.go：56 –-> FieldAccess: SameSite

代码片段：

```
53      MaxAge:   int(maxAge.Seconds()),
54      HttpOnly: true,
55      Secure:   CookieSecure(c),
56      SameSite: http.SameSiteLaxMode,
57   })
58}
```



北京启明星辰信息安全技术有限公司

2026-11-12